

PRESIDENCY UNIVERSITY

CSE322: Microprocessors Based System Design Laboratory

Class 02: SAP-1 Architecture & Instruction Cycle

Understanding a Simple Programmable Computer

Afzal Hossan

Lecturer

Department of Computer Science and Engineering

Presidency University

Lecture 02: Learning Objectives

By the end of today's lecture, students should be able to:

1. Explain the purpose and design philosophy of the **SAP-1** computer.
2. Identify the major components of SAP-1 and explain their roles.
3. Describe the SAP-1 common bus and data-flow organization.
4. Explain the functions of the accumulator, B register, PC, MAR, IR and output register.
5. Describe how the ALU and control unit cooperate during instruction execution.
6. Explain the SAP-1 instruction format and basic instruction set.
7. Trace the **fetch–decode–execute** cycle of a SAP-1 instruction.
8. Trace a simple SAP-1 program and determine the final output.

Connection to the Course

SAP-1 provides a small, transparent model for understanding how the registers, buses, ALU and control unit studied in Lecture 01 are combined to form a programmable computer.

Lecture 01

Registers	Bus	ALU	Control Unit



Lecture 02

How do these components form a working computer?

What is SAP-1?

SAP

SAP stands for **Simple-As-Possible**.

SAP-1 is a small educational computer architecture used to demonstrate the fundamental organization and operation of a processor.

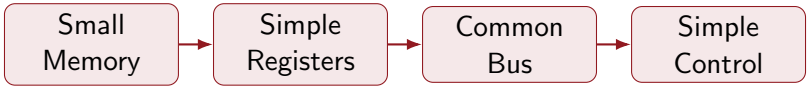
- Small memory and short instructions
- Simple register organization
- A common data bus
- A simple arithmetic/logic unit
- A hardwired-style control sequence
- A small instruction set

Why Study SAP-1?

The architecture is small enough that we can trace an instruction from memory to the control unit, through the ALU, and finally to an output.

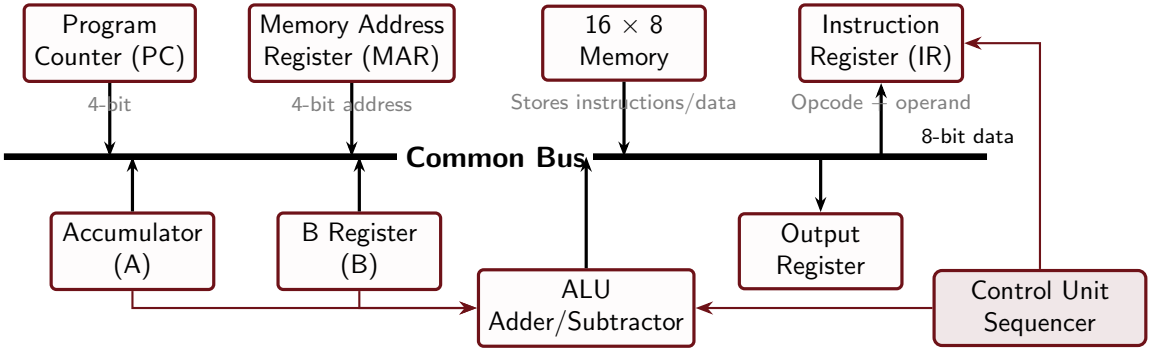
SAP-1 Design Philosophy

SAP-1 intentionally removes many features found in modern processors.



- The goal is **clarity**, not performance.
- The architecture exposes the basic data path.
- Instructions are intentionally simple.
- The design prepares us for the more sophisticated **8086**.

SAP-1 at a Glance



Key Idea

All major registers communicate through a common **8-bit bus**. The control unit determines which register places data on the bus and which register receives it during each clock step.

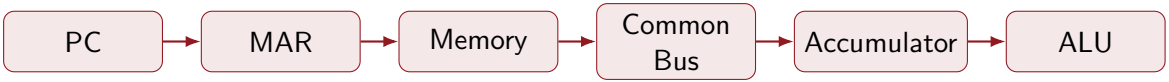
Major SAP-1 Components

Component	Main Function
Accumulator (A)	Stores an operand and receives the result of ALU operations.
B Register	Temporarily stores the second operand for arithmetic operations.
Program Counter (PC)	Holds the address of the next instruction to be fetched.
Memory Address Register (MAR)	Holds the memory address currently being accessed.
Instruction Register (IR)	Holds the current instruction being decoded/executed.
Output Register	Holds data that is sent to the output.
Memory	Stores program instructions and data.
ALU	Performs arithmetic operations on processor data.
Control Unit	Generates control signals that coordinate data movement and operations.
Common Bus	Provides a shared path for transferring data among compo-

SAP-1 Data Path

The SAP-1 data path can be viewed as:

$$\text{Registers} \longrightarrow \text{Bus} \longrightarrow \text{Registers / ALU}$$

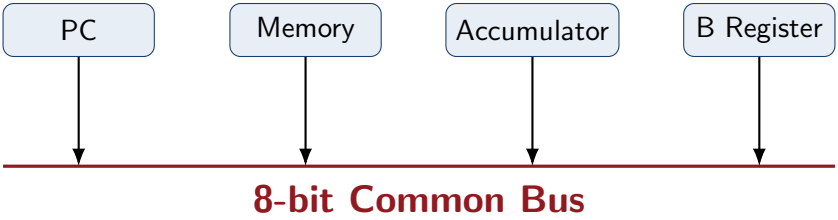


Important

The bus is a **shared communication path**; it does not itself perform arithmetic or store instructions permanently.

The 8-bit Common Bus

SAP-1 uses a common bus for transferring 8-bit information.



- One selected source places data on the bus.
- A destination receives the bus value when enabled.
- Control signals determine who may place data on the bus.

Key Principle

At a given transfer step, the control unit must coordinate the source and destination so that unintended simultaneous transfers do not occur.

Program Counter (PC)

Definition

The **Program Counter** stores the address of the next instruction to be fetched from memory.

In SAP-1, the PC is commonly represented as a **4-bit counter**.

$$\boxed{PC \in \{0000_2, \ldots, 1111_2\}}$$

Therefore, the address space contains:

$$2^4 = 16$$

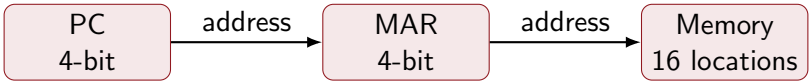
addressable memory locations.

Memory Address Register (MAR)

The **Memory Address Register** stores the address of the memory location being accessed.

$$PC \longrightarrow MAR \longrightarrow Memory$$

- The PC identifies the next instruction.
- The MAR holds the address used for the memory access.
- The memory uses that address to select a location.



SAP-1 Memory Organization

A simple SAP-1 memory organization can be represented as:

Address	8-bit Contents
0000	instruction / data
0001	instruction / data
0010	instruction / data
0011	instruction / data
⋮	⋮
1111	instruction / data

Capacity

$$16 \text{ locations} \times 8 \text{ bits} = 128 \text{ bits}$$

of storage.

Accumulator and B Register

Accumulator (A)

The accumulator is the main working register for arithmetic operations.
Typical role:

$$A \leftarrow A + B$$

or

$$A \leftarrow A - B$$

B Register

The B register temporarily holds the second operand.
For example:

$$A = 00000101$$

$$B = 00000011$$

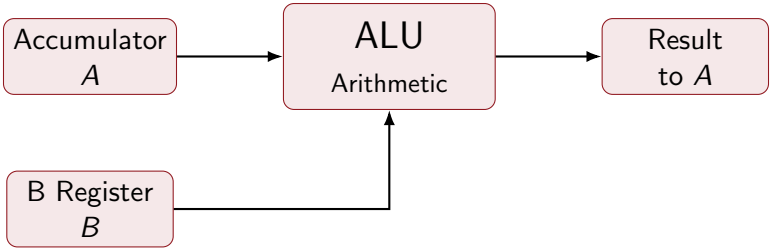
Then the ALU can operate on:

$$A + B$$

$$\boxed{A,\ B \longrightarrow ALU \longrightarrow A}$$

Arithmetic Logic Unit (ALU)

The ALU is the computational part of the SAP-1 data path.



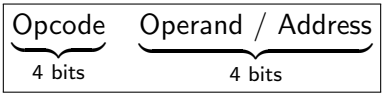
For a simplified SAP-1 arithmetic operation:

$$\boxed{A \leftarrow A + B}$$

The control unit determines when the required registers and ALU operation are activated.

Instruction Register (IR)

The **Instruction Register** holds the current instruction. A basic SAP-1 instruction can be represented using:



Therefore:

8-bit instruction

Example

Suppose an instruction is:

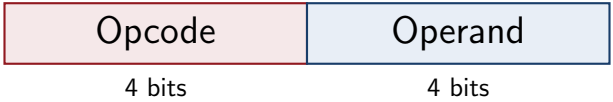
0001 1010

Then:

0001 = opcode

1010 = operand/address

SAP-1 Instruction Format



- **Opcode**: identifies the operation.
- **Operand**: usually identifies a memory address for instructions that require one.

Why 4 + 4 bits?

A 4-bit opcode field allows up to

$$2^4 = 16$$

different opcode patterns, while the 4-bit operand can address

$$2^4 = 16$$

memory locations.

Basic SAP-1 Instruction Set

A commonly used SAP-1 instruction set contains the following basic operations:

Instruction	Operation	Meaning
LDA	$A \leftarrow M[address]$	Load memory data into the accumulator.
ADD	$A \leftarrow A + M[address]$	Add memory data to the accumulator.
SUB	$A \leftarrow A - M[address]$	Subtract memory data from the accumulator.
OUT	$OUT \leftarrow A$	Copy accumulator contents to output.
HLT	Stop	Halt program execution.

Observation

Only a few instructions are enough to demonstrate the essential fetch–decode–execute mechanism of a processor.

Understanding LDA

LDA means **Load Accumulator**.
Suppose:

$$M[9] = 00001101_2$$

and the instruction is:

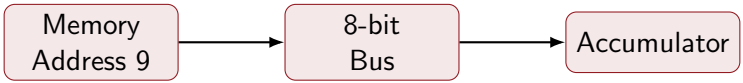
$$LDA\ 9$$

Then:

$$\boxed{A \leftarrow M[9]}$$

Therefore:

$$A = 00001101_2$$



Understanding ADD

Suppose:

$$A = 00000101_2$$

and

$$M[10] = 00000011_2$$

The instruction is:

$$ADD\ 10$$

Then:

$$A \leftarrow A + M[10]$$

Therefore:

$$A = 00000101_2 + 00000011_2 = 00001000_2$$

Important

The memory value is brought to the B register and the ALU performs the addition with the accumulator.

Understanding SUB

Suppose:

$$A = 00001000_2$$

and

$$M[11] = 00000010_2$$

The instruction is:

$$SUB\ 11$$

Then:

$$A \leftarrow A - M[11]$$

Therefore:

$$A = 00001000_2 - 00000010_2 = 00000110_2$$

The result is stored back in the accumulator.

$$\boxed{A,\ B \longrightarrow ALU\ (-) \longrightarrow A}$$

Understanding OUT and HLT

OUT

The **OUT** instruction transfers the accumulator contents to the output register.

$$OUT \Rightarrow OUT_{reg} \leftarrow A$$

The output register then holds the value to be observed externally.

HLT

The **HLT** instruction tells the control system to stop normal instruction execution.

$$HLT \Rightarrow \text{Stop}$$

$$\text{Program} \rightarrow \text{Compute} \rightarrow OUT \rightarrow HLT$$

Instruction Cycle in SAP-1

Every instruction follows the fundamental sequence:

$$\text{FETCH} \rightarrow \textit{DECODE} \rightarrow \textit{EXECUTE}$$

1. Fetch

Obtain the next instruction from memory using the address in the PC.

2. Decode

Interpret the opcode in the instruction register.

3. Execute

Perform the operation specified by the instruction.

Fetch Cycle: Conceptual Data Flow

Suppose:

$$PC = 0011_2$$

The processor wants to fetch the instruction stored at address 3.

$$\boxed{PC \longrightarrow MAR \longrightarrow Memory \longrightarrow Bus \longrightarrow IR}$$

Conceptually:

1. Copy the PC value to the MAR.
2. Use MAR to select the memory location.
3. Place the memory word on the bus.
4. Load the instruction into the IR.
5. Advance the PC.

Decode Cycle

After fetching:

$$IR = \boxed{\text{Opcode} \mid \text{Operand}}$$

The control unit examines the opcode.



The control unit determines:

- which register should output data,
- which register should receive data,
- whether the ALU should add or subtract,
- and when each operation should occur.

Execute Cycle

The execute phase depends on the instruction.

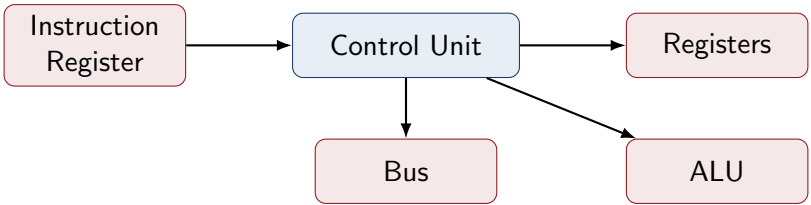
Instruction	Conceptual Execute Operation
LDA	Memory operand $\rightarrow$ Bus $\rightarrow$ Accumulator
ADD	Memory operand $\rightarrow$ B register; ALU adds $A + B$; result $\rightarrow A$
SUB	Memory operand $\rightarrow$ B register; ALU subtracts $A - B$; result $\rightarrow A$
OUT	Accumulator $\rightarrow$ Output Register
HLT	Stop normal instruction execution

Central Idea

The instruction does not magically execute in one indivisible action. The control unit coordinates a sequence of smaller register-transfer operations.

Control Unit: The Coordinator

The control unit converts an instruction into a sequence of control actions.



Think of it this way

ALU = performs the operation
Control Unit = tells the processor when and how to perform it

Clock and Timing

Processor operations are coordinated by clock pulses.

$$\boxed{T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow \cdots}$$

At different timing steps, different control signals are activated.

Timing step	Example activity
T_1	Place the current instruction address into the MAR.
T_2	Perform the memory access / update the instruction-fetch path.
T_3	Transfer the fetched instruction into the IR.
T_4	Begin the instruction-specific execute sequence.

Worked Example: SAP-1 Program Execution

Memory

0	*LDA* 9
1	*ADD* 10
2	*SUB* 11
3	*OUT*
4	*HLT*
9	5
10	3
11	2

Worked Example: SAP-1 Program Execution

Memory

0	*LDA* 9
1	*ADD* 10
2	*SUB* 11
3	*OUT*
4	*HLT*
9	5
10	3
11	2

Execution Trace

$PC = 0, \quad A = 0$

Worked Example: SAP-1 Program Execution

Memory

0	*LDA* 9
1	*ADD* 10
2	*SUB* 11
3	*OUT*
4	*HLT*
9	5
10	3
11	2

Execution Trace

$$PC = 0, \quad A = 0$$

$$LDA\ 9 \quad : \quad A = 5$$

Worked Example: SAP-1 Program Execution

Memory

0	*LDA* 9
1	*ADD* 10
2	*SUB* 11
3	*OUT*
4	*HLT*
9	5
10	3
11	2

Execution Trace

$$PC = 0, \quad A = 0$$

$$\begin{aligned} \textit{LDA}\ 9 &: A = 5 \\ \textit{ADD}\ 10 &: A = 5 + 3 = \boxed{8} \end{aligned}$$

Worked Example: SAP-1 Program Execution

Memory

0	*LDA* 9
1	*ADD* 10
2	*SUB* 11
3	*OUT*
4	*HLT*
9	5
10	3
11	2

Execution Trace

$$PC = 0, \quad A = 0$$

$$\begin{aligned} \textit{LDA}\ 9 &: A = 5 \\ \textit{ADD}\ 10 &: A = 5 + 3 = \boxed{8} \\ \textit{SUB}\ 11 &: A = 8 - 2 = \boxed{6} \end{aligned}$$

Worked Example: SAP-1 Program Execution

Memory

0	*LDA* 9
1	*ADD* 10
2	*SUB* 11
3	*OUT*
4	*HLT*
9	5
10	3
11	2

Execution Trace

$$PC = 0, \quad A = 0$$

$$\begin{aligned} \textit{LDA } 9 &: A = 5 \\ \textit{ADD } 10 &: A = 5 + 3 = \boxed{8} \\ \textit{SUB } 11 &: A = 8 - 2 = \boxed{6} \\ \textit{OUT} &: \boxed{6} \end{aligned}$$

Worked Example: SAP-1 Program Execution

Memory

0	*LDA* 9
1	*ADD* 10
2	*SUB* 11
3	*OUT*
4	*HLT*
9	5
10	3
11	2

Execution Trace

$$PC = 0, \quad A = 0$$

$$\begin{aligned} \textit{LDA } 9 &: A = 5 \\ \textit{ADD } 10 &: A = 5 + 3 = \boxed{8} \\ \textit{SUB } 11 &: A = 8 - 2 = \boxed{6} \\ \textit{OUT} &: \boxed{6} \\ \textit{HLT} &: \textit{STOP} \end{aligned}$$

Worked Example: SAP-1 Program Execution

Memory

0	*LDA* 9
1	*ADD* 10
2	*SUB* 11
3	*OUT*
4	*HLT*
9	5
10	3
11	2

Execution Trace

$$PC = 0, \quad A = 0$$

$$\begin{aligned} \textit{LDA } 9 &: A = 5 \\ \textit{ADD } 10 &: A = 5 + 3 = \boxed{8} \\ \textit{SUB } 11 &: A = 8 - 2 = \boxed{6} \\ \textit{OUT} &: \boxed{6} \\ \textit{HLT} &: \textit{STOP} \end{aligned}$$

Final Output = 6

Worked Example: Instruction Trace

Step	PC	Instruction	Accumulator	Output
1	0	LDA 9	5	–
2	1	ADD 10	8	–
3	2	SUB 11	6	–
4	3	OUT	6	6
5	4	HLT	6	6

What should students observe?

- The PC identifies which instruction comes next.
- The accumulator stores intermediate results.
- Memory supplies program instructions and operands.
- The ALU performs arithmetic.
- OUT makes the accumulator value observable.

Tracing LDA Step-by-Step

For:

$$LDA\ 9$$

assume:

$$PC = 0, \qquad M[0] = LDA\ 9, \qquad M[9] = 00000101_2$$

1. **Fetch:** PC identifies address 0.
2. **MAR:** address 0 is placed in MAR.
3. **Memory read:** $M[0]$ provides the instruction.
4. **IR:** $LDA\ 9$ is loaded into the IR.
5. **Decode:** control unit recognizes LDA.
6. **Operand address:** address 9 is used to access memory.
7. **Load:** $M[9]$ is transferred to the accumulator.

Therefore:

Tracing ADD Step-by-Step

For:

$$ADD\ 10$$

assume:

$$A = 00000101_2, \qquad M[10] = 00000011_2$$

Conceptually:

$$M[10] \rightarrow B$$

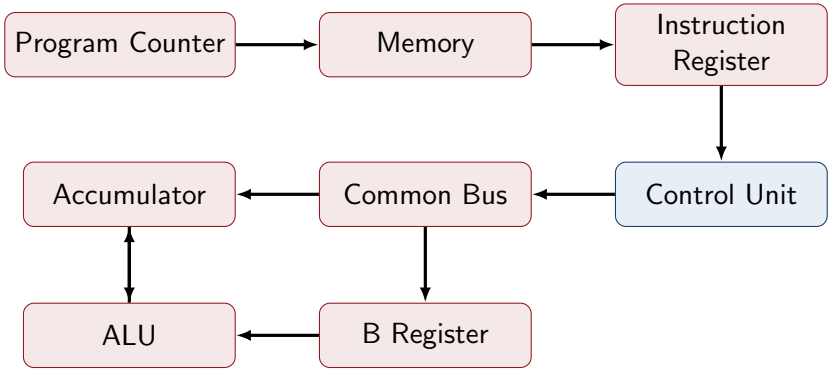
then:

$$A = 00000101_2$$

$$B = 00000011_2$$

$$A + B = 00000101_2 + 00000011_2 = 00001000_2$$

SAP-1 Instruction Execution: Big Picture



Fetch → Decode → Execute

SAP-1: What It Teaches Us

SAP-1 makes several fundamental processor concepts visible:

1. A processor needs **storage** — registers.
2. It needs a **communication path** — bus.
3. It needs a **computational unit** — ALU.
4. It needs **control** — control unit.
5. It needs **memory** to store instructions and data.
6. It needs a **program counter** to sequence instructions.
7. Instructions are executed through a sequence of coordinated micro-operations.

Conceptual Bridge

$$\boxed{\text{SAP-1} \longrightarrow \text{8086} \longrightarrow \text{Modern Processors}}$$

The complexity increases, but the fundamental ideas remain recognizable.

SAP-1 vs. 8086

Feature	SAP-1	8086
Design purpose	Educational	General-purpose microprocessor
Data organization	Simple 8-bit path	16-bit architecture
Memory scale	Very small	Much larger address space
Instruction set	Very small	Extensive
Registers	Few and simple	Multiple programmer-visible registers
Addressing	Simple	Multiple addressing modes
Control	Simple	Much more sophisticated
I/O and system support	Minimal	Extensive

Why learn SAP-1 first?

SAP-1 lets us understand the **principles** before dealing with the larger register set, segmented memory, instruction set and bus organization of the 8086.

Limitations of SAP-1

SAP-1 is intentionally limited.

- Very small memory.
- Very small instruction set.
- Limited arithmetic capability.
- Simple addressing mechanism.
- Minimal I/O capability.
- No sophisticated interrupt architecture.
- No complex addressing modes.
- Not intended for real-world high-performance computing.

Important Distinction

These limitations are not design failures. They make the architecture small enough to study processor organization at the register-transfer level.

Common Misconceptions

1. **"The bus stores data."**
   The bus primarily provides a communication path.
2. **"The PC contains the current instruction."**
   The PC identifies the address of the next instruction to fetch; the IR holds the current instruction.
3. **"The ALU controls the processor."**
   The ALU performs computation; the control unit coordinates operations.
4. **"An instruction is always one hardware action."**
   An instruction can require several smaller register-transfer operations.
5. **"Memory and registers are the same."**
   Registers are processor storage elements; memory is a separate storage system accessed through the processor's data path.

Classroom Exercise 1: Identify the Component

Match each operation with the appropriate component.

Operation	Component
Stores the address of a memory access	?
Stores the next instruction address	?
Stores the current instruction	?
Performs addition/subtraction	?
Holds the second arithmetic operand	?
Provides the shared data path	?
Generates coordination signals	?
Holds the value sent to output	?

Think first; answer before looking at the next slide.

Classroom Exercise 1: Answer

Operation	Answer
Stores the address of a memory access	MAR
Stores the next instruction address	PC
Stores the current instruction	IR
Performs addition/subtraction	ALU
Holds the second arithmetic operand	B Register
Provides the shared data path	Common Bus
Generates coordination signals	Control Unit
Holds the value sent to output	Output Register

Classroom Exercise 2: Trace the Program

Consider:

Address	Contents
0	LDA 9
1	ADD 10
2	OUT
3	HLT
9	7
10	4

Answer the following:

1. What is the value of the accumulator after `LDA 9`?
2. What is the value of the accumulator after `ADD 10`?
3. What value appears in the output register after `OUT`?
4. At which instruction does execution stop?

Do the trace manually before discussing the answer.

Classroom Exercise 2: Answer

$$LDA\ 9 \Rightarrow A = 7$$

Then:

$$ADD\ 10 \Rightarrow A = 7 + 4 = 11$$

Then:

$$OUT \Rightarrow OUT_{reg} = 11$$

Finally:

$$HLT \Rightarrow \text{execution stops}$$

Therefore:

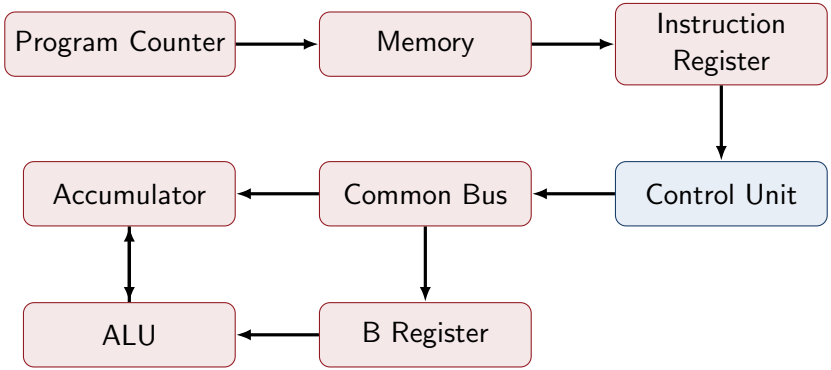
$$\boxed{\text{Final accumulator} = 11}$$

$$\boxed{\text{Final output} = 11}$$

Key Takeaways

1. SAP-1 means **Simple-As-Possible** and is an educational processor architecture.
2. The main components are registers, memory, bus, ALU and control unit.
3. The PC identifies the next instruction address.
4. The MAR holds a memory address during an access.
5. The IR holds the current instruction.
6. The accumulator and B register support arithmetic operations.
7. The ALU performs arithmetic operations.
8. The control unit coordinates processor activity.
9. SAP-1 uses a simple opcode/operand instruction format.
10. Instructions follow the fundamental **fetch–decode–execute** cycle.

Concept Map


Program Counter
Memory
Instruction Register
Accumulator
Common Bus
Control Unit
ALU
B Register

Next: SAP-1 Memory Organization & Operation

We will go deeper into:

- SAP-1 memory organization
- Address and data flow
- Instruction execution in greater detail
- Timing and control signals
- Designing and tracing a simple SAP-1 program
- Limitations of the SAP-1 architecture

Preparation

Review:

Registers	Buses	ALU	Fetch–Decode–Execute

Quick Review Questions

1. What does SAP stand for?
2. Why is SAP-1 useful for learning processor organization?
3. What is the purpose of the PC?
4. What is the purpose of the MAR?
5. What does the IR contain?
6. Why is a common bus used?
7. What is the role of the B register?
8. What does the ALU do?
9. What does the control unit do?
10. What are the three fundamental stages of instruction execution?
11. What is the difference between an opcode and an operand?
12. What happens when an OUT instruction is executed?

Thank You

Questions?

CSE322: Microprocessors Based System Design Laboratory

Afzal Hossan
Lecturer, Department of Computer Science and Engineering
Presidency University